

An infinite range concept

Document #: P3555R0
Date: 2025-01-13
Project: Programming Language C++
Audience: SG9
Reply-to: Jonathan Müller (think-cell)
<foonathan@jonathanmueller.dev>

Contents

- 1 Abstract
- 2 Motivation
 - 2.1 Statically catch buggy infinite loops
 - 2.2 Parallel binary transform
 - 2.3 Bounds checks elimination
- 3 Prior art
- 4 Proposed design
 - 4.1 The `ranges::infinite_range` concept
 - 4.1.1 Option A: `ranges::enable_infinite_range`
 - 4.1.2 Option B: Tag type returned by `.size()` or ADL `size()`
 - 4.1.3 Option A vs. Option B
 - 4.1.4 Option C: Like option B, but change `ranges::size`
 - 4.2 Propagating cardinality in range factories
 - 4.3 Propagating cardinality in range adaptors
 - 4.3.1 `join_view` and `join_with_view`
 - 4.3.2 `concat_view`
 - 4.3.3 `zip_view` and `zip_transform_view`
 - 4.4 Using `ranges::infinite_range`
 - 4.4.1 Statically catch buggy infinite loops
 - 4.4.2 Parallel binary transform
 - 4.4.3 Bounds check elimination
- 5 Wording
- 6 References

§ 1 Abstract

We propose a new concept `ranges::infinite_range` to statically detect infinite ranges. This makes it possible to statically catch buggy infinite loops, resolving [LWG4019], allows for a common idiom with parallel algorithms, improving [P3179R4], and enables optimizations to eliminate unnecessary bounds checks, like the manual one proposed in [P3230R1].

§ 2 Motivation

§ 2.1 Statically catch buggy infinite loops

[LWG4019] points out that the following code contains an infinite loop:

```
auto a = views::iota(0) | views::reverse;  
a.begin(); // infinite loop
```

The problem is that `views::iota` is a non-common range. Therefore, the implementation of `reverse_view::begin()` has to manually find the end of `views::iota` before going backwards from there. But `views::iota` is infinite, so there is no end, and it loops forever.

The issue submitter proposes a detection of `unreachable_sentinel_t` in `views::reverse`: By definition, ranges with that sentinel type are infinitely sized, and passing them to `views::reverse` should not compile. However, this solution is unsatisfactory as there are more infinite ranges not covered by the detection of `unreachable_sentinel_t` (e.g. `views::zip(views::iota(0), views::iota(1))`). We therefore need a more general mechanism to detect infinite ranges to properly resolve [LWG4019]. This sentiment was shared by a joint SG9/LWG meeting in St. Louis.

§ 2.2 Parallel binary transform

Another motivation comes from [P3179R4], proposing rangified algorithms with execution policies. One of them is binary transform:

```
ranges::transform(execution::par, rng1, rng2, output, fn);
```

The implementation needs to know the size of the input ranges to properly partition the input for parallel processing. The paper originally proposed that it is enough if one of `rng1` and `rng2` is sized, and the algorithm should then assume that the other one is at least as big. However, SG9 decided in Wrocław that both ranges must be sized for safety reasons.

This decision was met with strong vendor resistance. A common idiom is to pass some sort of index or constant sequence as `rng1` and then some data as `rng2`. The natural way, passing the indexes as an infinite range, does not work, because infinite ranges aren't sized:

```
ranges::transform(execution::par, views::iota(0), data, output, fn); // infinite range  
is not sized
```

In theory, this is fine; the implementation only requires knowing `min(size(rng1), size(rng2))` which in the case of an infinite range is just `size(rng2)`. But we again need a mechanism to detect infinite ranges to implement that.

§ 2.3 Bounds checks elimination

Finally, [P3230R1] proposes `views::unchecked_take` and `views::unchecked_drop`. They work just like `views::take` and `views::drop`, but assume the range is bigger than the number of elements taken or dropped, avoiding bounds checks and improving performance. The common case of `infinite_range`

| `views::take(n)` could be automatically optimized to `infinite_range` |
`views::unchecked_take(n)` — if there is a way to statically detect infinite ranges. Likewise, bounds checks in other range adaptors could be eliminated.

A mechanism to detect infinite ranges can enable these optimizations.

§ 3 Prior art

[[range-v3](#)] has the concept of a *range cardinality*. A range can have one of the following cardinalities:

Known finite size

the range is finite and the size is known (e.g. a vector)

Unknown finite size

the range is finite but the size is not known (e.g. a `views::filter` of a vector)

Infinite size

the range is infinite (e.g. `views::repeat`)

Unknown

the size of the range is completely unknown, including whether it is finite or infinite (e.g. a generator)

This information is then propagated by (some) views. For example, `transform` leaves the cardinality unchanged, and a `filter` of a finite range stays finite but a `filter` of an infinite range has an unknown cardinality.

This design was dropped during standardization due to implementation complexity for little benefit.

§ 4 Proposed design

We propose a simplified model of cardinality. In our proposal, a range can have only the following cardinalities:

Known finite size

corresponding to the `ranges::sized_range`

Infinite size

corresponding to the newly proposed `ranges::infinite_range`

Unknown

if neither concept is satisfied

Compared to the [[range-v3](#)] model, a range can no longer have an unknown but finite size. This means that not `ranges::infinite_range` does not mean “finite range” it means “finite range or infinite range that we couldn’t detect”. For example, `views::filter` can be finite (when filtering a finite range) or infinite (when filtering an infinite range and keeping an infinite subset of elements). However, when it is finite, we do not know what size it is, so it cannot model `ranges::sized_range`.

The benefit of our proposal is reduced implementation complexity; fewer views need to propagate cardinality information than in the `range-v3` model. We also believe that knowing that a range is definitely finite is not particularly useful. The inverse, knowing that a range is definitely infinite, is enough to catch infinite loops and omit bounds checks.

§ 4.1 The `ranges::infinite_range` concept

Whether a range is infinite is a semantic, not a syntactic distinction, which requires manual opt-in or opt-out. There is one exception: if the sentinel type of a range is `unreachable_sentinel_t`, then the range is definitely infinite.

For other ranges, we propose an opt-in: a range does not model `ranges::infinite_range` by default. There are two obvious ways to design the opt-in.

§ 4.1.1 Option A: `ranges::enable_infinite_range`

Similar to `ranges::enable_borrowed_range`, there could be a variable template `ranges::enable_infinite_range` that can be specialized to indicate that a type models `ranges::infinite_range`. The primary specialization checks whether the sentinel is `unreachable_sentinel_t`; the concept delegates:

```
namespace std::ranges {
    template <class R>
        constexpr bool enable_infinite_range = same_as<sentinel_t<R>,
            unreachable_sentinel_t>;

    template <class R>
        concept infinite_range = range<R> && enable_infinite_range<remove_cvref_t<R>>;
}
```

User code can then opt-in by specializing `ranges::enable_infinite_range`:

```
template <>
constexpr bool std::ranges::enable_infinite_range<my_infinite_range> = true;

template <class R>
constexpr bool
    std::ranges::enable_infinite_range<my_range_adaptor_that_does_not_affect_cardinality<R>>
    = std::ranges::infinite_range<R>;
```

§ 4.1.2 Option B: Tag type returned by `.size()` or ADL `size()`

Alternatively, we could leverage the fact that a range is either infinite or sized but never both, and have `size()` return a tag type to indicate infinite. Note that it is a designated tag *type* not a special tag *value* like `std::size_t(-1)` as we need to distinguish it at compile-time. Let's call it `infinite_tag_t` for now (there is an argument to be made for re-using `unreachable_sentinel_t`):

```
namespace std { // or std::ranges
    struct infinite_tag_t {};
    inline constexpr infinite_tag_t infinite_tag;
}
```

`ranges::infinite_range` is then modeled if and only if `size()` returns that tag type. Like `ranges::size` we check for member and non-member `size()`; unlike `ranges::size` we don't need to worry about arrays (definitely finite), random access ranges with sized sentinel (definitely finite), or `disable_sized_range` (if you return an `infinite_tag_t` you better use the semantics we want!).

```
namespace std::ranges {
    template <class R>
        concept infinite_range = range<R> && (
            same_as<sentinel_t<R>, unreachable_sentinel_t>
            || requires(R&& r) { { r.size() } -> same_as<infinite_tag_t>; }
            || requires(R&& r) { { /*ADL*/size(r) } -> same_as<infinite_tag_t>; }
        );
}
```

Note that a type that models `ranges::infinite_range` can never model `ranges::sized_range`, which requires that `size()` returns an integer type. Also note that in this option we do not propose changing `std::ranges::size`; it will never return `infinite_tag_t`.

Users can then opt-in by returning `std::infinite_tag` from `size()`:

```
class my_infinite_range {
public:
    auto size() const { return std::infinite_tag; }
};

template <class R>
class my_range_adaptor_that_does_not_affect_cardinality {
    R _r;

public:
    auto size() const requires std::ranges::sized_range<R> { return
std::ranges::size(_r); }
    auto size() const requires std::ranges::infinite_range<R> { return
std::infinite_tag; }
};
```

§ 4.1.3 Option A vs. Option B

The example code for `my_range_adaptor_that_does_not_affect_cardinality` shows one potential advantage of option B over option A: In many cases, a range adaptor wants to both propagate the size (if there is one) and the infinite-ness; see below for a list of those range adaptors. With option B and some helper code, it is possible to do this without a lot of code duplication.

```
namespace detail {
    auto potentially_infinite_size(std::ranges::sized_range auto&& r) { return
std::ranges::size(r); }
    auto potentially_infinite_size(std::ranges::infinite_range auto&&) { return
std::infinite_tag; }

    template <class R>
    concept enable_size = requires(R&& r) { detail::potentially_infinite_size(r); };
}
```

Changing a range adaptor from propagating only size to propagating size and infinite-ness is then trivial:

```
template <class R>
class my_range_adaptor_that_does_not_affect_cardinality {
    R _r;

public:
-     auto size() const requires std::ranges::sized_range<R> { return
std::ranges::size(_r); }
+     auto size() const requires detail::enable_size<R> { return
detail::potentially_infinite_size(_r); }
};
```

With option A, we would need to additionally write a separate specialization for `enable_infinite_range`.

The need for a `potentially_infinite_size()` function then makes one wonder: what if `ranges::size` could also return that?

§ 4.1.4 Option C: Like option B, but change `ranges::size`

Under this option, we still have the `infinite_tag_t`, but we relax `std::ranges::size`: When it delegates to `.size()` or `ADL-size()`, it also allows `infinite_tag_t`. We also add a case to return `infinite_tag`:

[range.prim.size]

- ² Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

- If `T` is an array of unknown bound ([dcl.array]), `ranges::size(E)` is ill-formed.
- Otherwise, if `T` is an array type, `ranges::size(E)` is expression-equivalent to `auto(extent_v<T>)`.
- Otherwise, if `disable_sized_range<remove_cv_t<T>>` ([range.sized]) is false and `auto(t.size())` is a valid expression of integer-like type ([iterator.concept.winc]) or of type `infinite_tag_t`, `ranges::size(E)` is expression-equivalent to `auto(t.size())`.
- Otherwise, if `T` is a class or enumeration type, `disable_sized_range<remove_cv_t<T>>` is false and `auto(size(t))` is a valid expression of integer-like type or of type `infinite_tag_t` where the meaning of `size` is established as-if by performing argument-dependent lookup only ([basic.lookup.argdep]), then `ranges::size(E)` is expression-equivalent to that expression.
- Otherwise, if `to-unsigned-like(ranges::end(t) - ranges::begin(t))` ([ranges.syn]) is a valid expression and the types `I` and `S` of `ranges::begin(t)` and `ranges::end(t)` (respectively) model both `sized_sentinel_for<S, I>` ([iterator.concept.sizedsentinel]) and `forward_iterator<I>`, then `ranges::size(E)` is expression-equivalent to `to-unsigned-like(ranges::end(t) - ranges::begin(t))`.
- Otherwise, if `same_as<sentinel_t<T>, unreachable_sentinel_t>` is true, `ranges::size(E)` is expression-equivalent to `infinite_tag`.
- Otherwise, `ranges::size(E)` is ill-formed.

`ranges::sized_range` then additionally checks for a finite size:

```
template <class T>
concept sized_range = range<T> && requires(T& t) { requires is-integer-like<decltype(ranges::size(t))>; };
```

For `infinite_range`, `ranges::size` has to return `infinite_tag_t`:

```
template <class T>
concept infinite_range = range<T> && requires(T& t) { { ranges::size(t) } -> same_as<infinite_tag_t>; };
```

This simplifies the conditional opt-in:

```
template <class R>
class my_range_adaptor_that_does_not_affect_cardinality {
    R _r;

public:
    - auto size() const requires std::ranges::sized_range<R> { return
      std::ranges::size(_r); }
```

```
+ auto size() const requires requires(R&& r) { std::ranges::size(r); } { return
    std::ranges::size(_r); }
};
```

Option C is not a breaking change: We only changed cases where `ranges::size` would have been ill-formed before. However, we break assumptions about an equivalence between `ranges::sized_range` and `ranges::size`. Badly constrained generic code might have called `ranges::size` unconditionally. But again, this just means trading one error message (`ranges::size` does not exist) to another one (`infinite_tag_t` is not an integer).

However, re-using `ranges::size` like that might be confusing.

§ 4.2 Propagating cardinality in range factories

`empty_view` and `single_view` are definitely finite, so don't need to change. Unbounded `iota_view` and `repeat_view` are already covered as they use `unreachable_sentinel_t`. `istream_view` and `generator` have an unknown cardinality, so they can never model `ranges::infinite_range`. We therefore do not propose any changes here.

`subrange` is interesting: If the sentinel type is `unreachable_sentinel_t`, it will model `ranges::infinite_range` automatically. Otherwise, the cardinality is unknown and cannot be easily derived as we lost information about the range type the iterators came from. We could extend the `ranges::subrange_kind` enumeration to add an additional `subrange_kind::infinite` value. Similar to `subrange_kind::unsized` `subrange` it would not store a size value, but it would unconditionally model `ranges::infinite_range`. As a result, constructing a `subrange_kind::infinite` `subrange` has a precondition that the range is actually infinite. Moreover, this precondition is safety critical as an implementation might choose to elide bounds checks for infinite range access. We therefore do not propose any changes to `subrange` in this paper; that option can always be explored later.

§ 4.3 Propagating cardinality in range adaptors

Note: This is meant to be an exhaustive list. If a range adaptor is not mentioned here, we have forgotten about it.

The following range adaptors model `ranges::infinite_range` if and only if their underlying range models `ranges::infinite_range`:

- `ref_view`
- `owning_view`
- `as_rvalue_view`
- `transform_view`
- `drop_view`
- `lazy_split_view`
- `split_view`
- `common_view`
- `reverse_view`
- `as_const_view`
- `elements_view`

- `enumerate_view`
- `adjacent_view`
- `adjacent_transform_view`
- `chunk_view`
- `slide_view`
- `chunk_by_view`
- `stride_view`
- `cache_latest_view`

We propose that they get the appropriate opt-in.

The following range adaptors are definitely finite or have an unknown cardinality:

- `filter_view` (unknown cardinality)
- `take_view` (finite)
- `take_while_view` (unknown cardinality)
- `drop_while_view` (unknown cardinality)

We thus do not propose any changes for them.

The remaining range adaptors require more detailed discussion.

§ 4.3.1 `join_view` and `join_with_view`

We propose that `join_view` models `ranges::infinite_range` if and only if the underlying outer range models `ranges::infinite_range` (joining infinitely many things together is definitely infinite). Note that the cardinality is unknown if only the underlying inner range models `ranges::infinite_range`: The outer range could be empty in which case `join_view` is empty.

If we have a mechanism to statically determine whether a range is non-empty, we could have `join_view` model `ranges::infinite_range` in more cases. However, we do not propose such a mechanism at this time.

The same logic applies to `join_with_view`. Note that there it is also not enough to only know that the separator models `ranges::infinite_range` for the same reason.

§ 4.3.2 `concat_view`

We propose that `concat_view` models `ranges::infinite_range` if at least one of the underlying ranges models `ranges::infinite_range`.

§ 4.3.3 `zip_view` and `zip_transform_view`

We propose that `zip_view` and `zip_transform_view` model `ranges::infinite_range` if all of the underlying ranges model `ranges::infinite_range`.

§ 4.4 Using `ranges::infinite_range`

§ 4.4.1 Statically catch buggy infinite loops

Once `ranges::infinite_range` exist, we can statically catch buggy infinite loops. Technically, an implementation is allowed to do that as a matter of QoI due to the guarantees in [intro.progress]: If an algorithm or view has an infinite loop without side-effects, the behavior is undefined, and undefined behavior can be lifted into compile-time errors.

However, given that an issue was already filed for `views::reverse`, we should mandate a detection at least for that view adaptor. We therefore propose that `views::reverse` is ill-formed for ranges modeling `ranges::infinite_range` but not also `ranges::common_range`, resolving [LWG4019].

Preventing infinite ranges in other situations requires implementation experience and has to be done more careful. For example, passing an infinite range to `std::ranges::for_each` is fine if the predicate has side effects, but calling `std::ranges::sort` on it is an error (unless the user does something funky with the comparison or swap). We therefore do not propose additional infinite loop detection.

§ 4.4.2 Parallel binary transform

As [P3179R4] is still in-flight, we do not propose any changes to it in this paper. We expect harmonization to appear in a follow-up paper.

§ 4.4.3 Bounds check elimination

Optimizations to eliminate bounds checks for infinite ranges are QoI and do not need to be mandated. We expect vendors to implement them once the concept is available.

§ 5 Wording

TBD

§ 6 References

[LWG4019] Barry Revzin. Reversing an infinite range leads to an infinite loop.

<https://wg21.link/lwg4019>

[P3179R4] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2024-12-11. C++ parallel range algorithms.

<https://wg21.link/p3179r4>

[P3230R1] Hewill Kang. 2024-12-01. `views::unchecked_(take|drop)`.

<https://wg21.link/p3230r1>

[range-v3] range-v3.

<https://github.com/ericniebler/range-v3>